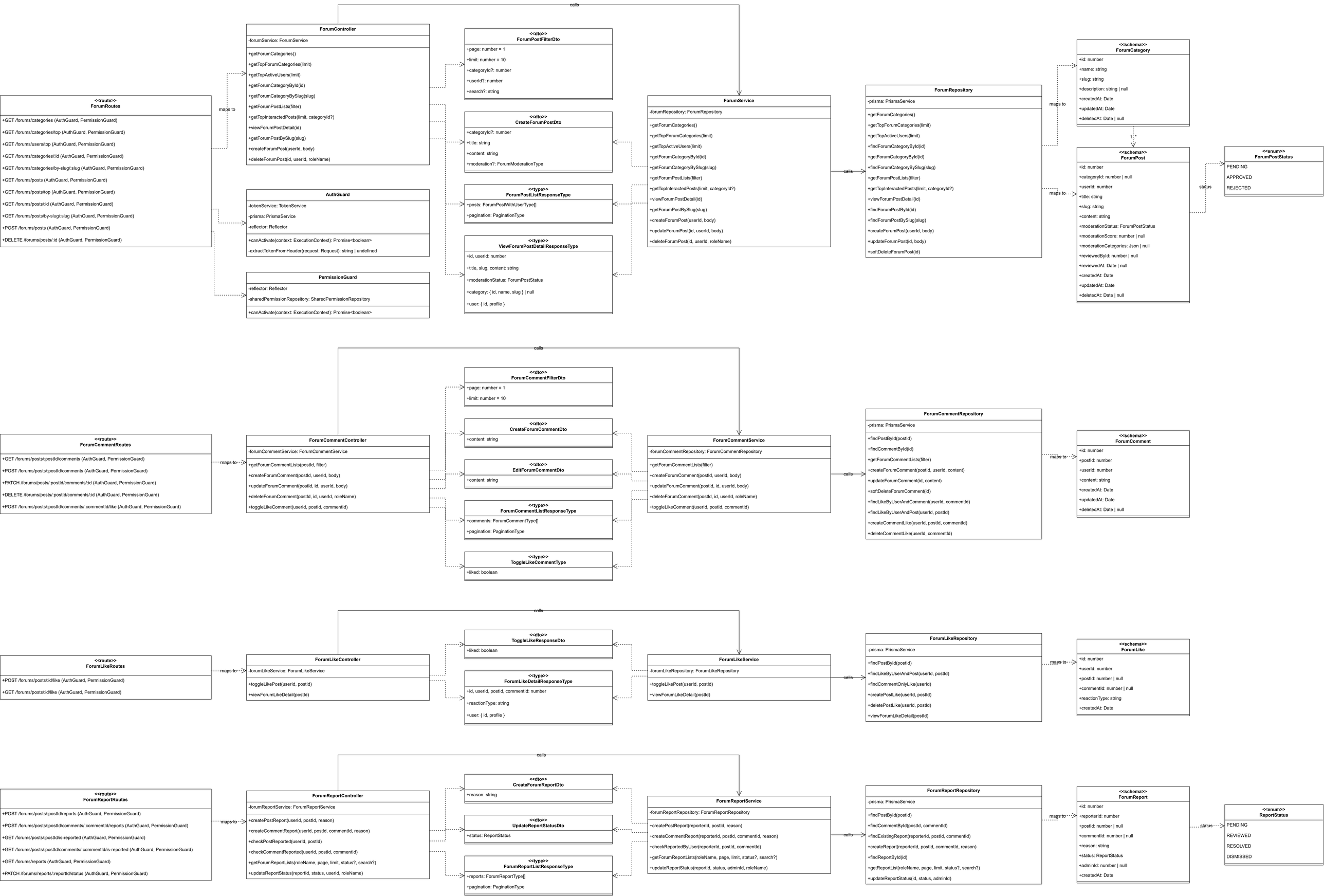

calls
ForumController
-forumService: ForumService
+getForumCategories()
+getTopForumCategories(limit)
+getTopActiveUsers(limit)
+getForumCategoryById(id)
+getForumCategoryBySlug(slug)
+getForumPostLists(filter)
+getTopInteractedPosts(limit, categoryId?)
+viewForumPostDetail(id)
+getForumPostBySlug(slug)
+createForumPost(userId, body)
+deleteForumPost(id, userId, roleName)
<<dto>>
ForumPostFilterDto
+page: number = 1
+limit: number = 10
+categoryId?: number
+userId?: number
+search?: string
<<dto>>
CreateForumPostDto
+categoryId?: number
+title: string
+content: string
+moderation?: ForumModerationType
<<type>>
ForumPostListResponseType
+posts: ForumPostWithUserType[]
+pagination: PaginationType
<<type>>
ViewForumPostDetailResponseType
+id, userId: number
+title, slug, content: string
+moderationStatus: ForumPostStatus
+category: { id, name, slug } | null
+user: { id, profile }
<<route>>
ForumRoutes
+GET /forums/categories (AuthGuard, PermissionGuard)
+GET /forums/categories/top (AuthGuard, PermissionGuard)
+GET /forums/users/top (AuthGuard, PermissionGuard)
+GET /forums/categories/:id (AuthGuard, PermissionGuard)
+GET /forums/categories/by-slug/:slug (AuthGuard, PermissionGuard)
+GET /forums/posts (AuthGuard, PermissionGuard)
+GET /forums/posts/top (AuthGuard, PermissionGuard)
+GET /forums/posts/:id (AuthGuard, PermissionGuard)
+GET /forums/posts/by-slug/:slug (AuthGuard, PermissionGuard)
+POST /forums/posts (AuthGuard, PermissionGuard)
+DELETE /forums/posts/:id (AuthGuard, PermissionGuard)
maps to
AuthGuard
-tokenService: TokenService
-prisma: PrismaService
-reflector: Reflector
+canActivate(context: ExecutionContext): Promise<boolean>
-extractTokenFromHeader(request: Request): string | undefined
PermissionGuard
-reflector: Reflector
-sharedPermissionRepository: SharedPermissionRepository
+canActivate(context: ExecutionContext): Promise<boolean>
ForumService
-forumRepository: ForumRepository
+getForumCategories()
+getTopForumCategories(limit)
+getTopActiveUsers(limit)
+getForumCategoryById(id)
+getForumCategoryBySlug(slug)
+getForumPostLists(filter)
+getTopInteractedPosts(limit, categoryId?)
+viewForumPostDetail(id)
+getForumPostBySlug(slug)
+createForumPost(userId, body)
+updateForumPost(id, userId, body)
+deleteForumPost(id, userId, roleName)
calls
ForumRepository
-prisma: PrismaService
+getForumCategories()
+getTopForumCategories(limit)
+getTopActiveUsers(limit)
+findForumCategoryById(id)
+getForumCategoryById(id)
+findForumCategoryBySlug(slug)
+getForumPostLists(filter)
+getTopInteractedPosts(limit, categoryId?)
+viewForumPostDetail(id)
+findForumPostById(id)
+findForumPostBySlug(slug)
+createForumPost(userId, body)
+updateForumPost(id, body)
+softDeleteForumPost(id)
maps to
<<schema>>
ForumCategory
+id: number
+name: string
+slug: string
+description: string | null
+createdAt: Date
+updatedAt: Date
+deletedAt: Date | null
1
*
<<schema>>
ForumPost
+id: number
+categoryId: number | null
+userId: number
+title: string
+slug: string
+content: string
+moderationStatus: ForumPostStatus
+moderationScore: number | null
+moderationCategories: Json | null
+reviewedById: number | null
+reviewedAt: Date | null
+createdAt: Date
+updatedAt: Date
+deletedAt: Date | null
maps to
status
<<enum>>
ForumPostStatus
PENDING
APPROVED
REJECTED
calls
ForumCommentController
-forumCommentService: ForumCommentService
+getForumCommentLists(postId, filter)
+createForumComment(postId, userId, body)
+updateForumComment(postId, id, userId, body)
+deleteForumComment(postId, id, userId, roleName)
+toggleLikeComment(userId, postId, commentId)
<<dto>>
ForumCommentFilterDto
+page: number = 1
+limit: number = 10
<<dto>>
CreateForumCommentDto
+content: string
<<dto>>
EditForumCommentDto
+content: string
<<type>>
ForumCommentListResponseType
+comments: ForumCommentType[]
+pagination: PaginationType
<<type>>
ToggleLikeCommentType
+liked: boolean
<<route>>
ForumCommentRoutes
+GET /forums/posts/:postId/comments (AuthGuard, PermissionGuard)
+POST /forums/posts/:postId/comments (AuthGuard, PermissionGuard)
+PATCH /forums/posts/:postId/comments/:id (AuthGuard, PermissionGuard)
+DELETE /forums/posts/:postId/comments/:id (AuthGuard, PermissionGuard)
+POST /forums/posts/:postId/comments/:commentId/like (AuthGuard, PermissionGuard)
maps to
ForumCommentService
-forumCommentRepository: ForumCommentRepository
+getForumCommentLists(filter)
+createForumComment(postId, userId, body)
+updateForumComment(postId, id, userId, body)
+deleteForumComment(postId, id, userId, roleName)
+toggleLikeComment(userId, postId, commentId)
calls
ForumCommentRepository
-prisma: PrismaService
+findPostById(postId)
+findCommentById(id)
+getForumCommentLists(filter)
+createForumComment(postId, userId, content)
+updateForumComment(id, content)
+softDeleteForumComment(id)
+findLikeByUserAndComment(userId, commentId)
+findLikeByUserAndPost(userId, postId)
+createCommentLike(userId, postId, commentId)
+deleteCommentLike(userId, commentId)
maps to
<<schema>>
ForumComment
+id: number
+postId: number
+userId: number
+content: string
+createdAt: Date
+updatedAt: Date
+deletedAt: Date | null
calls
ForumLikeController
-forumLikeService: ForumLikeService
+toggleLikePost(userId, postId)
+viewForumLikeDetail(postId)
<<dto>>
ToggleLikeResponseDto
+liked: boolean
<<type>>
ForumLikeDetailResponseType
+id, userId, postId, commentId: number
+reactionType: string
+user: { id, profile }
<<route>>
ForumLikeRoutes
+POST /forums/posts/:id/like (AuthGuard, PermissionGuard)
+GET /forums/posts/:id/like (AuthGuard, PermissionGuard)
maps to
ForumLikeService
-forumLikeRepository: ForumLikeRepository
+toggleLikePost(userId, postId)
+viewForumLikeDetail(postId)
calls
ForumLikeRepository
-prisma: PrismaService
+findPostById(postId)
+findLikeByUserAndPost(userId, postId)
+findCommentOnlyLike(userId)
+createPostLike(userId, postId)
+deletePostLike(userId, postId)
+viewForumLikeDetail(postId)
maps to
<<schema>>
ForumLike
+id: number
+userId: number
+postId: number | null
+commentId: number | null
+reactionType: string
+createdAt: Date
calls
ForumReportController
-forumReportService: ForumReportService
+createPostReport(userId, postId, reason)
+createCommentReport(userId, postId, commentId, reason)
+checkPostReported(userId, postId)
+checkCommentReported(userId, postId, commentId)
+getForumReportLists(roleName, page, limit, status?, search?)
+updateReportStatus(reportId, status, userId, roleName)
<<dto>>
CreateForumReportDto
+reason: string
<<dto>>
UpdateReportStatusDto
+status: ReportStatus
<<type>>
ForumReportListResponseType
+reports: ForumReportType[]
+pagination: PaginationType
<<route>>
ForumReportRoutes
+POST /forums/posts/:postId/reports (AuthGuard, PermissionGuard)
+POST /forums/posts/:postId/comments/:commentId/reports (AuthGuard, PermissionGuard)
+GET /forums/posts/:postId/is-reported (AuthGuard, PermissionGuard)
+GET /forums/posts/:postId/comments/:commentId/is-reported (AuthGuard, PermissionGuard)
+GET /forums/reports (AuthGuard, PermissionGuard)
+PATCH /forums/reports/:reportId/status (AuthGuard, PermissionGuard)
maps to
ForumReportService
-forumReportRepository: ForumReportRepository
+createPostReport(reporterId, postId, reason)
+createCommentReport(reporterId, postId, commentId, reason)
+checkReportedByUser(reporterId, postId, commentId)
+getForumReportLists(roleName, page, limit, status?, search?)
+updateReportStatus(reportId, status, adminId, roleName)
calls
ForumReportRepository
-prisma: PrismaService
+findPostById(postId)
+findCommentById(postId, commentId)
+findExistingReport(reporterId, postId, commentId)
+createReport(reporterId, postId, commentId, reason)
+findReportById(id)
+getReportList(roleName, page, limit, status?, search?)
+updateReportStatus(id, status, adminId)
maps to
<<schema>>
ForumReport
+id: number
+reporterId: number
+postId: number | null
+commentId: number | null
+reason: string
+status: ReportStatus
+adminId: number | null
+createdAt: Date
status
<<enum>>
ReportStatus
PENDING
REVIEWED
RESOLVED
DISMISSED